



TECNOLOGICO NACIONAL DE MEXICO

INSTITUTO TECNOLÓGICO de Tuxtepec

“PRÁCTICAS EN PYTHON”

Materia:

Ciencia de Datos

Carrera

Ing. Sistemas Computacionales

Presenta:

**Manuel Vasquez Xochilt
22350343**

Asesor:

Dr. Julio Aguilar Carmona

Grado y grupo:

8° “A”

Tuxtepec, Oax.

20 de mayo de 2026



Práctica 2: Análisis de Calidad Industrial (Industria 4.0)

Código:

```
# Práctica 2: Análisis de Calidad Industrial
```

```
# Importar librerías
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Crear datos
```

```
np.random.seed(42)
```

```
n = 500
```

```
temperatura = np.random.normal(75, 5, n)
```

```
tasa_error = (temperatura * 0.3) + np.random.normal(0, 2, n)
```

```
turno = np.random.choice(['Matutino', 'Vespertino'], n)
```

```
df = pd.DataFrame({  
    'temperatura': temperatura,  
    'tasa_error': tasa_error,  
    'turno': turno  
})
```

```
# Gráfica 1
```

```
plt.boxplot([  
    df[df['turno'] == 'Matutino']['tasa_error'],  
    df[df['turno'] == 'Vespertino']['tasa_error']  
])
```

```
plt.xticks([1, 2], ['Matutino', 'Vespertino'])
```

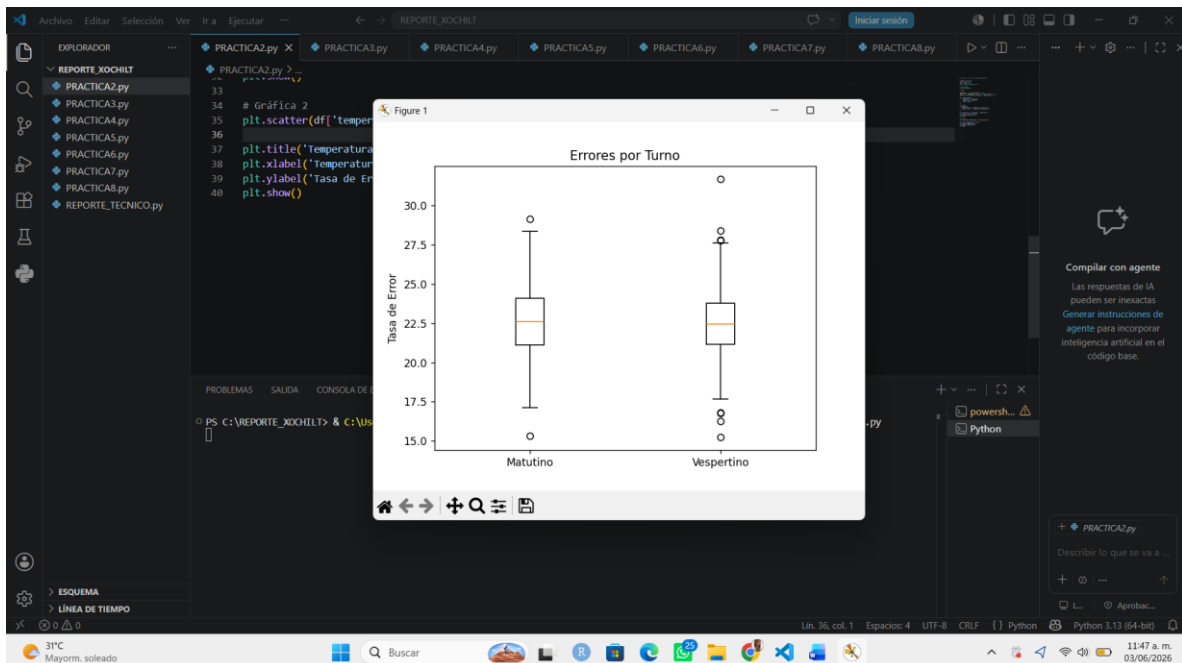
```
plt.title('Errores por Turno')  
plt.ylabel('Tasa de Error')  
plt.show()
```

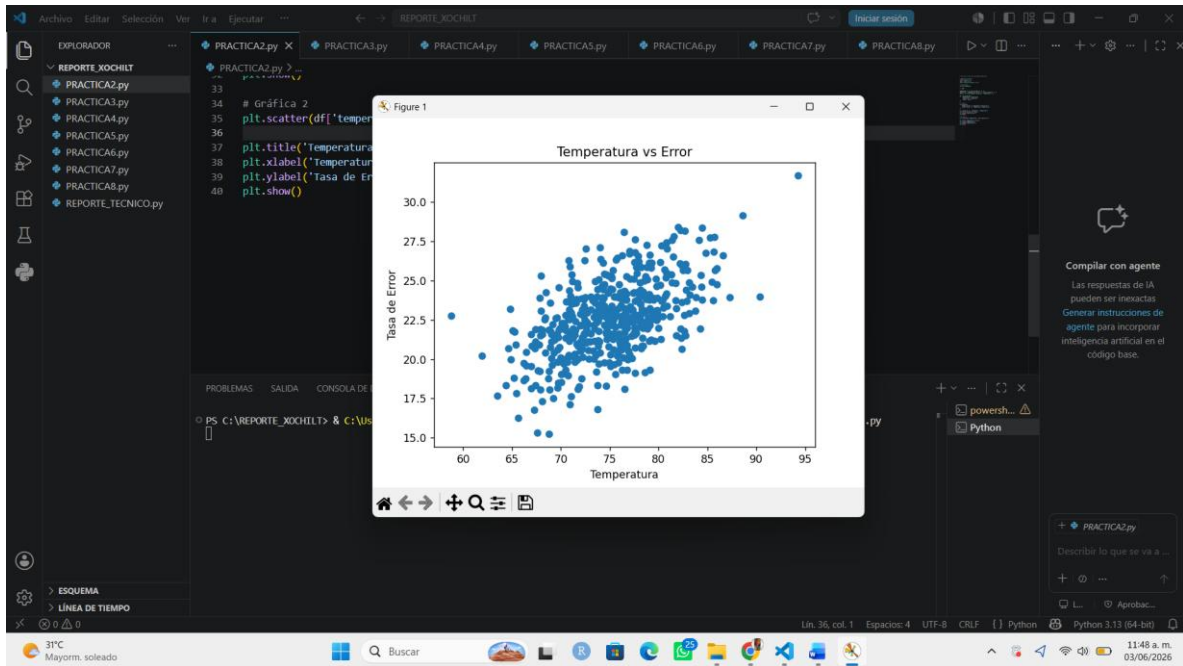
Gráfica 2

```
plt.scatter(df['temperatura'], df['tasa_error'])
```

```
plt.title('Temperatura vs Error')  
plt.xlabel('Temperatura')  
plt.ylabel('Tasa de Error')  
plt.show()
```

Resultado:





Práctica 3: Análisis de Eficiencia en Logística Global

Código:

```
# Práctica 3: Análisis de Eficiencia en Logística Global
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Crear datos
```

```
np.random.seed(42)
```

```
n = 600
```

```
distancia = np.random.normal(500, 120, n)
```

```
tiempo = distancia * 0.08 + np.random.normal(0, 3, n)
```

```
transporte = np.random.choice(['Terrestre', 'Aéreo'], n)
```

```
df = pd.DataFrame({  
    'distancia': distancia,  
    'tiempo': tiempo,  
    'transporte': transporte  
})
```

```
# Gráfica 1
```

```
for tipo in df['transporte'].unique():
```

```
    datos = df[df['transporte'] == tipo]
```

```
    plt.scatter(datos['distancia'], datos['tiempo'], label=tipo)
```

```
plt.title('Distancia y Tiempo')
```

```
plt.xlabel('Distancia')
```

```
plt.ylabel('Tiempo')
```

```
plt.legend()
```

```
plt.show()
```

Gráfica 2

```
plt.boxplot([
    df[df['transporte'] == 'Terrestre']['tiempo'],
    df[df['transporte'] == 'Aéreo']['tiempo']
])
```

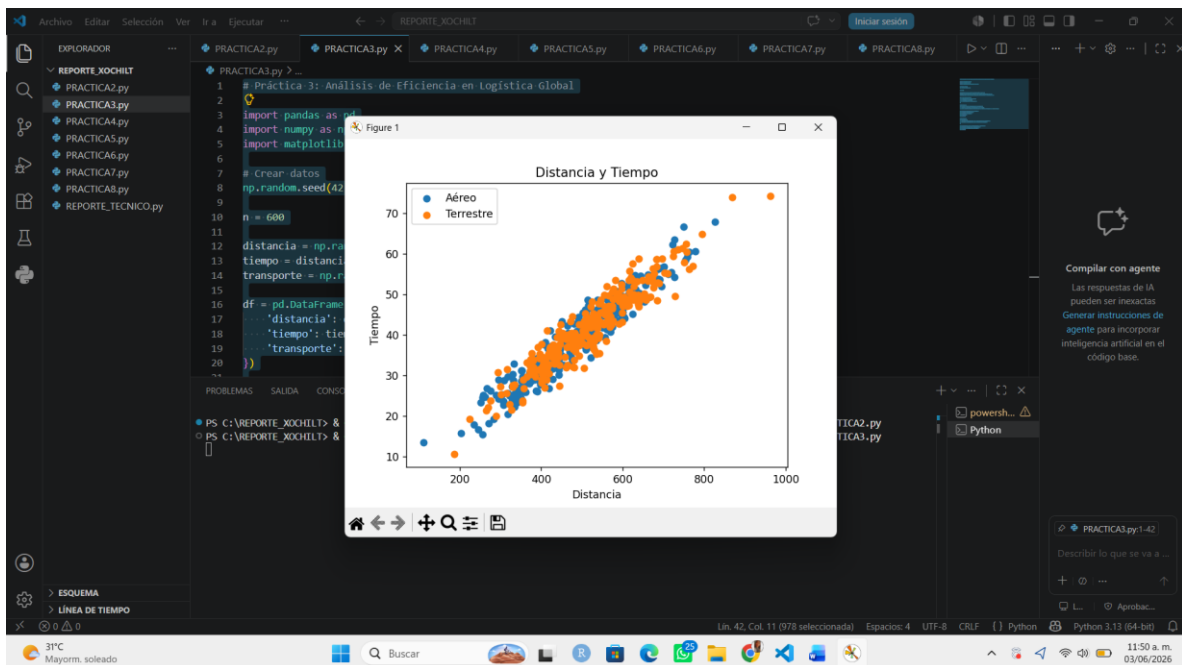
```
plt.xticks([1, 2], ['Terrestre', 'Aéreo'])
```

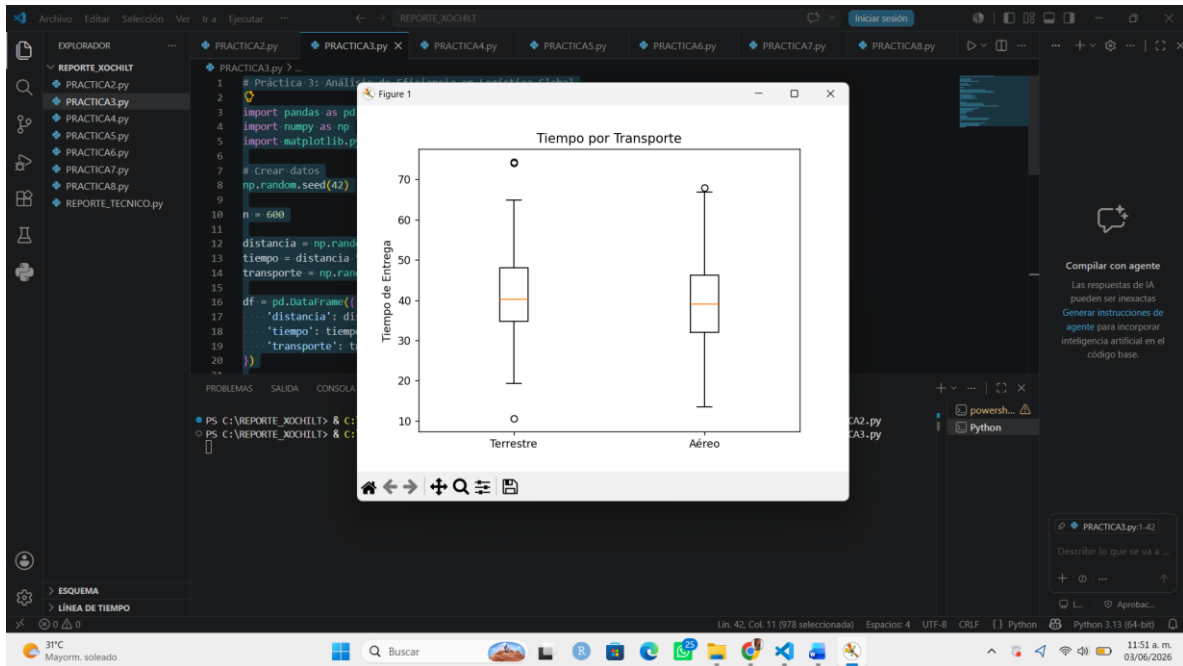
```
plt.title('Tiempo por Transporte')
```

```
plt.ylabel('Tiempo de Entrega')
```

```
plt.show()
```

Resultado:





Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial

Código:

```
# Práctica 4: Reducción de Dimensionalidad en Monitoreo Industrial
```

```
# Importar librerías
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Crear datos
```

```
np.random.seed(42)
```

```
n = 500
```

```
df = pd.DataFrame({  
    'temp': np.random.normal(80, 5, n),  
    'presion': np.random.normal(30, 3, n),  
    'vibracion': np.random.normal(10, 2, n),  
    'gas': np.random.normal(50, 8, n)  
})
```

```
# PCA
```

```
scaler = StandardScaler()
```

```
datos = scaler.fit_transform(df)
```

```
pca = PCA(n_components=2)
```



```
resultado = pca.fit_transform(datos)
```

```
# Gráfica 1
```

```
plt.plot(  
    np.cumsum(pca.explained_variance_ratio_),  
    marker='o'  
)
```

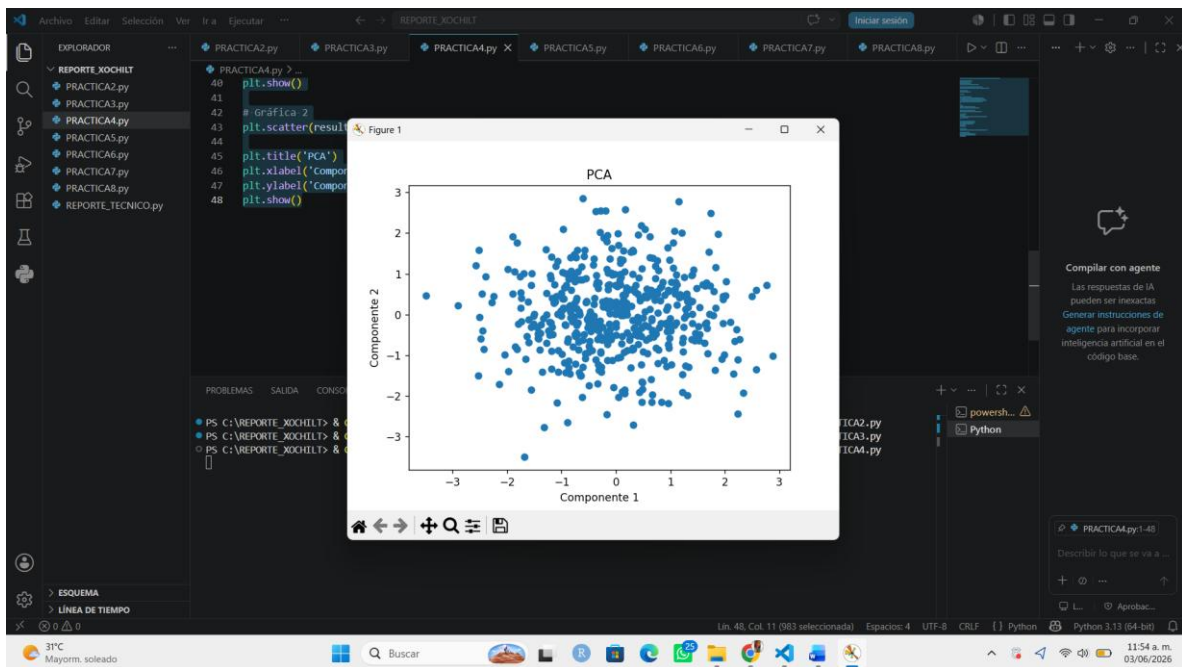
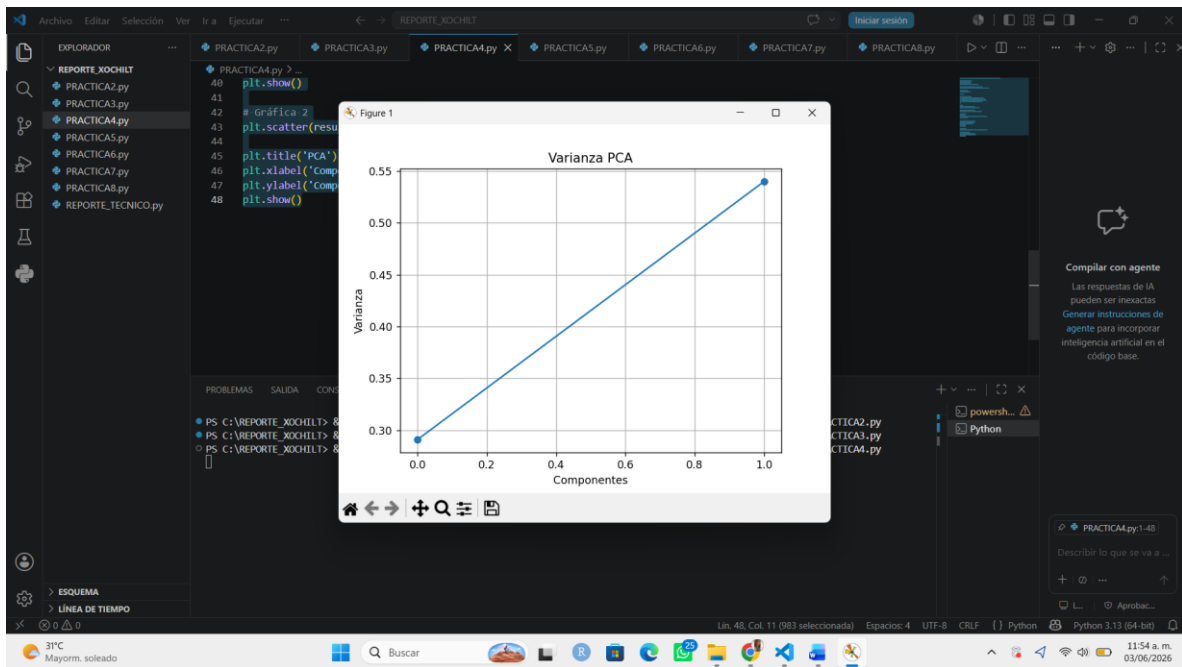
```
plt.title('Varianza PCA')  
plt.xlabel('Componentes')  
plt.ylabel('Varianza')  
plt.grid()  
plt.show()
```

```
# Gráfica 2
```

```
plt.scatter(resultado[:, 0], resultado[:, 1])
```

```
plt.title('PCA')  
plt.xlabel('Componente 1')  
plt.ylabel('Componente 2')  
plt.show()
```

Resultado:



Reporte Técnico: Reducción de Dimensionalidad en Monitoreo Industrial

Código:

```
# Reporte Técnico: Reducción de Dimensionalidad en Monitoreo Industrial
```

```
# Importar librerías
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Crear datos
```

```
np.random.seed(42)
```

```
n = 200
```

```
df = pd.DataFrame({  
    'temp1': np.random.normal(100, 5, n),  
    'temp2': np.random.normal(100, 5, n),  
    'presion': np.random.normal(50, 10, n),  
    'vibracion': np.random.normal(20, 3, n),  
    'ruido': np.random.normal(60, 5, n),  
    'voltaje': np.random.normal(220, 2, n)  
})
```

```
# PCA
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(df)
```

```
pca = PCA()
X_pca = pca.fit_transform(X)

# Gráfica 1 (varianza)
plt.plot(np.cumsum(pca.explained_variance_ratio_), marker='o')
plt.axhline(0.85, linestyle='--')

plt.title('Varianza acumulada')
plt.xlabel('Componentes')
plt.ylabel('Varianza')

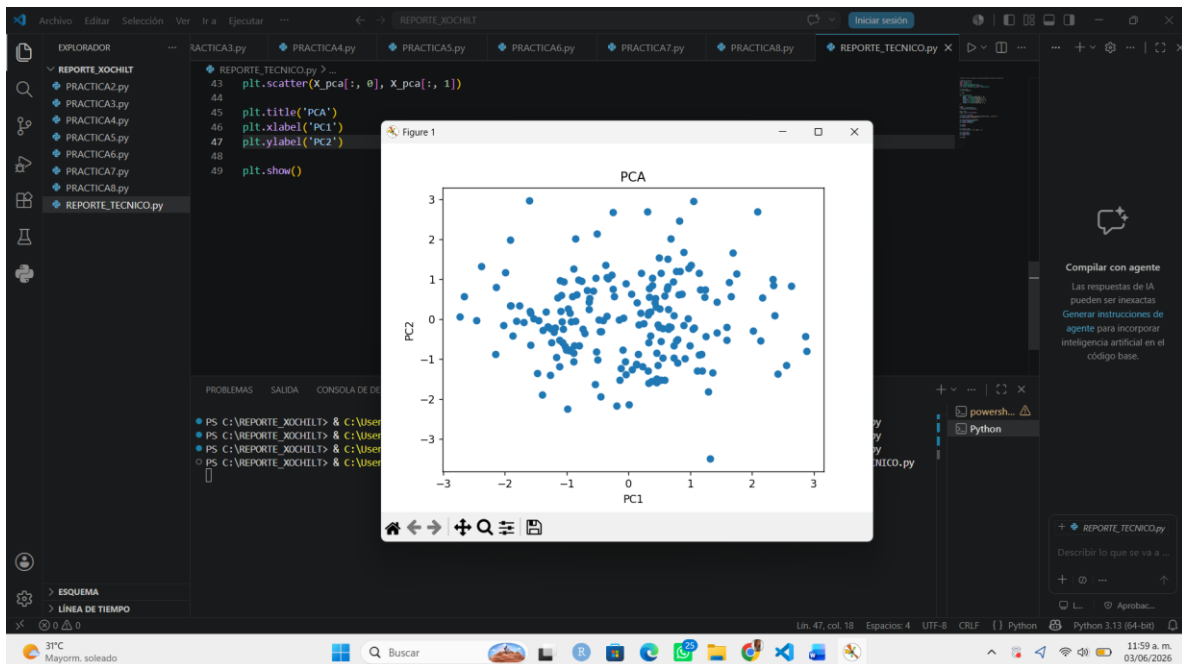
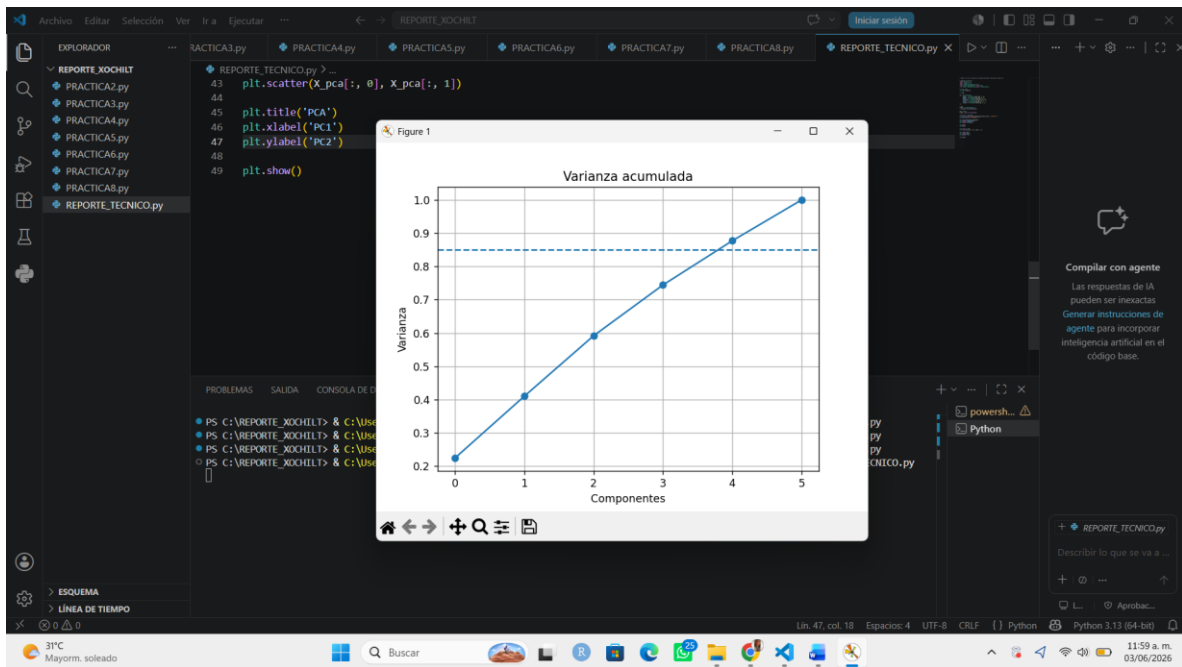
plt.grid()
plt.show()

# Gráfica 2 (PCA)
plt.scatter(X_pca[:, 0], X_pca[:, 1])

plt.title('PCA')
plt.xlabel('PC1')
plt.ylabel('PC2')

plt.show()
```

Resultado:



Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)

Código:

```
# Práctica 5: Reducción de Dimensiones en Tráfico de Red (Cyber-Systems)
```

```
# Importar librerías
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Crear datos
```

```
np.random.seed(123)
```

```
n = 300
```

```
df = pd.DataFrame({  
    'duracion': np.random.normal(50, 10, n),  
    'paquetes': np.random.normal(100, 20, n),  
    'errores': np.random.poisson(2, n),  
    'latencia': np.random.normal(15, 5, n),  
    'jitter': np.random.normal(2, 0.5, n),  
    'cpu': np.random.normal(40, 10, n),  
    'memoria': np.random.normal(40, 10, n),  
    'http': np.random.normal(200, 50, n)  
})
```

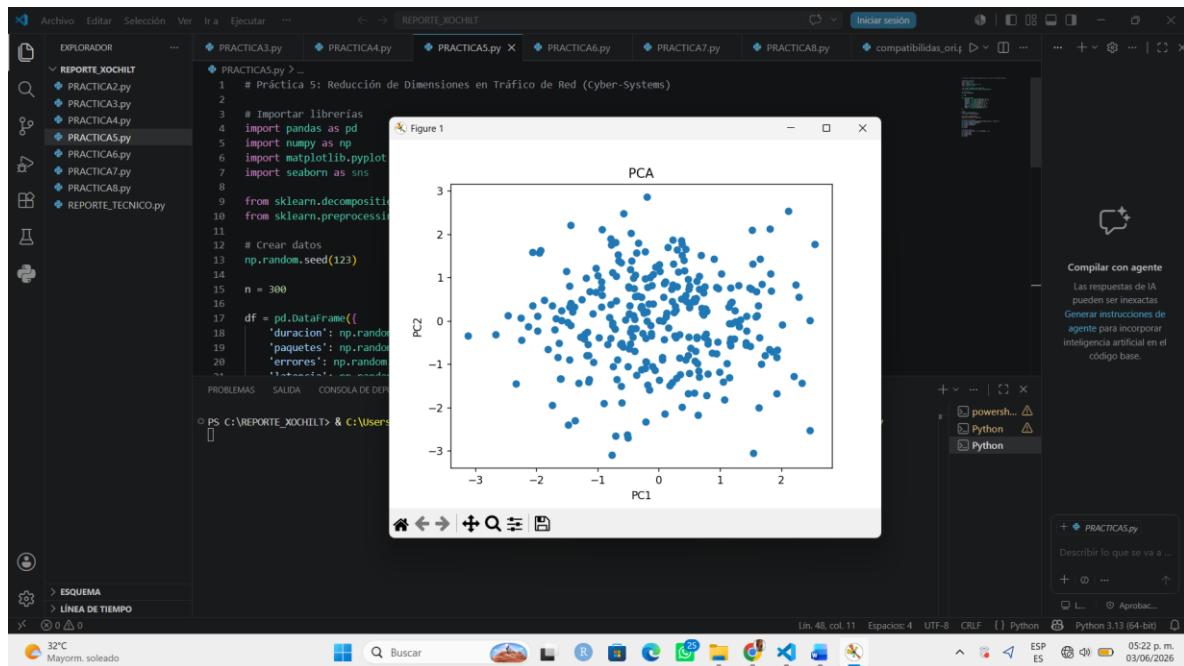
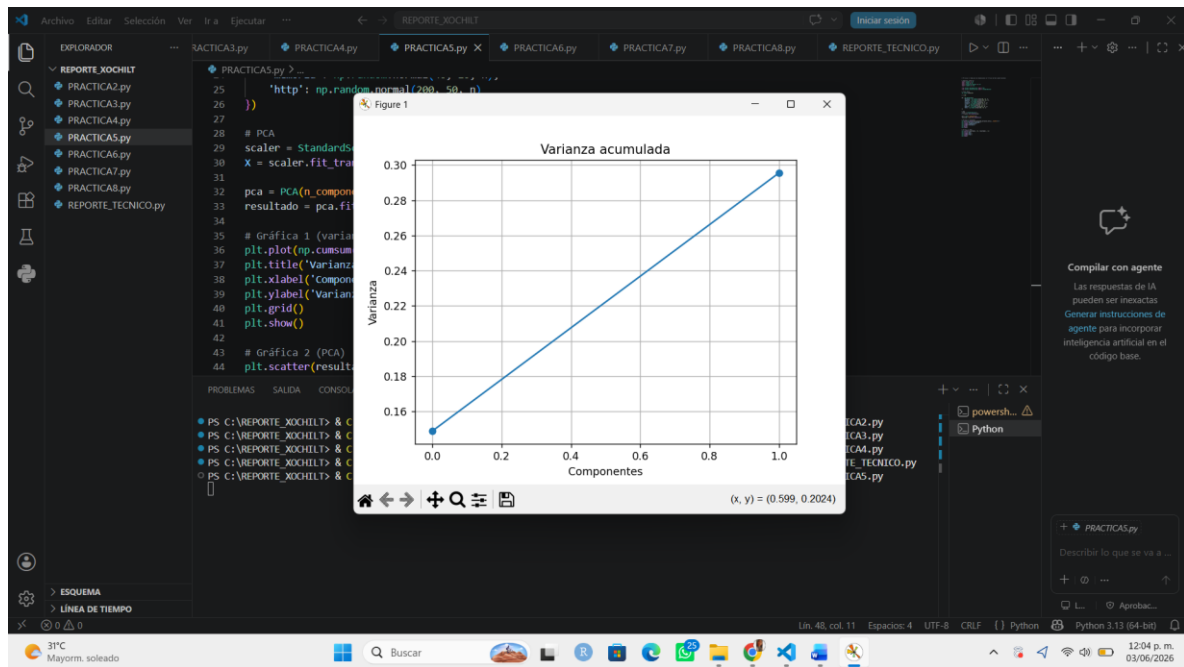
```
# PCA
scaler = StandardScaler()
X = scaler.fit_transform(df)

pca = PCA(n_components=2)
resultado = pca.fit_transform(X)

# Gráfica 1 (varianza)
plt.plot(np.cumsum(pca.explained_variance_ratio_), marker='o')
plt.title('Varianza acumulada')
plt.xlabel('Componentes')
plt.ylabel('Varianza')
plt.grid()
plt.show()

# Gráfica 2 (PCA)
plt.scatter(resultado[:, 0], resultado[:, 1])
plt.title('PCA')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Resultado:



Práctica 6: Optimización de Sensores en Smart Cities

Código:

```
# Práctica 6: Optimización de Sensores en Smart Cities
```

```
# Importar librerías
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Crear datos
```

```
np.random.seed(456)
```

```
n = 400
```

```
df = pd.DataFrame({  
    'temp': np.random.normal(28, 4, n),  
    'humedad': np.random.normal(60, 10, n),  
    'vehiculos': np.random.normal(120, 30, n),  
    'viento': np.random.normal(15, 5, n),  
    'solar': np.random.normal(800, 100, n),  
    'peatones': np.random.normal(50, 15, n)  
})
```

```
# PCA
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(df)
```

```
pca = PCA(n_components=2)
resultado = pca.fit_transform(X)
```

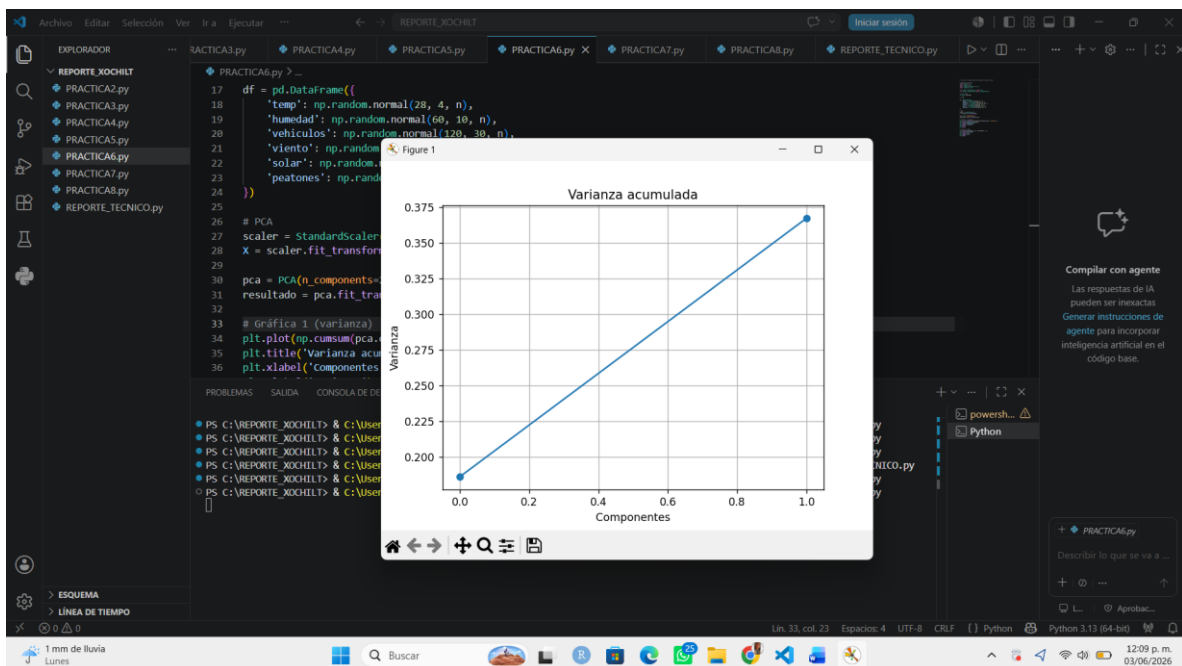
```
# Gráfica 1 (varianza)
```

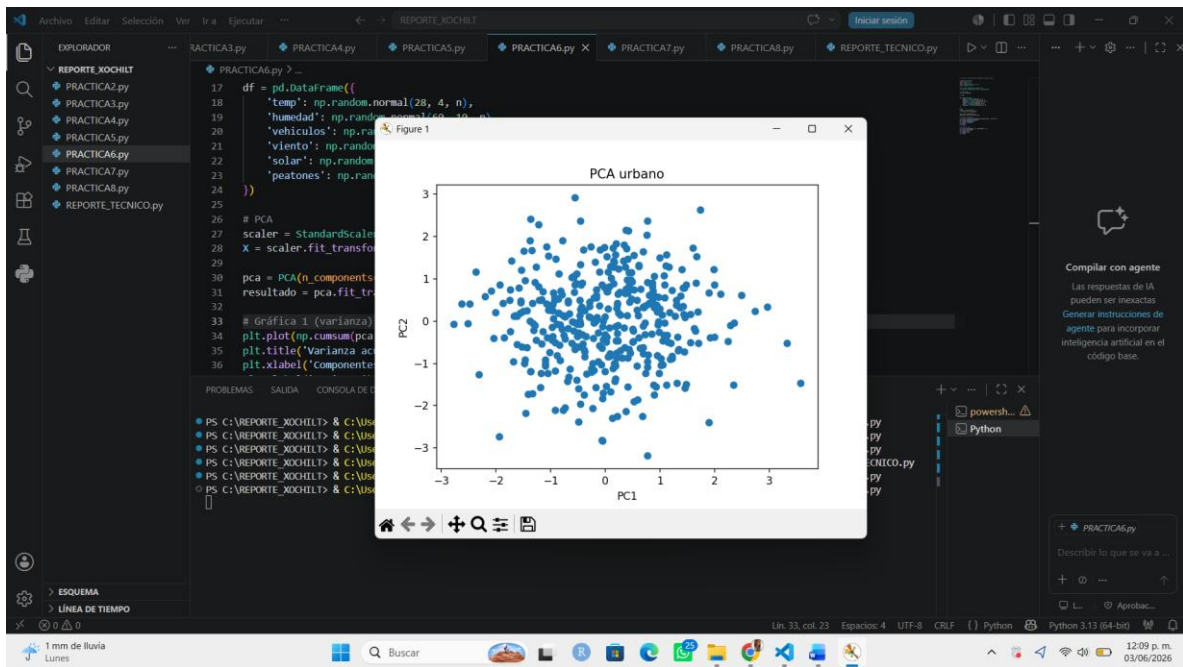
```
plt.plot(np.cumsum(pca.explained_variance_ratio_), marker='o')
plt.title('Varianza acumulada')
plt.xlabel('Componentes')
plt.ylabel('Varianza')
plt.grid()
plt.show()
```

```
# Gráfica 2 (PCA)
```

```
plt.scatter(resultado[:, 0], resultado[:, 1])
plt.title('PCA urbano')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```

Resultado:





Práctica 7: Implementación de Modelos de Machine Learning

Código:

```
# Práctica 7: Implementación de Modelos de Machine Learning
```

```
# Importar librerías
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.cluster import KMeans
```

```
from sklearn.preprocessing import StandardScaler
```

```
# Crear datos
```

```
np.random.seed(789)
```

```
n = 500
```

```
df = pd.DataFrame({  
    'experiencia': np.random.normal(5, 2, n),  
    'certificaciones': np.random.poisson(3, n),  
    'habilidades': np.random.uniform(1, 10, n),  
    'salario': np.random.normal(50000, 10000, n)  
})
```

```
# K-Means
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(df[['experiencia', 'salario']])
```

```
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
df['cluster'] = kmeans.fit_predict(X)
```

```
# Gráfica única
```

```
plt.scatter(df['experiencia'], df['salario'], c=df['cluster'])
```

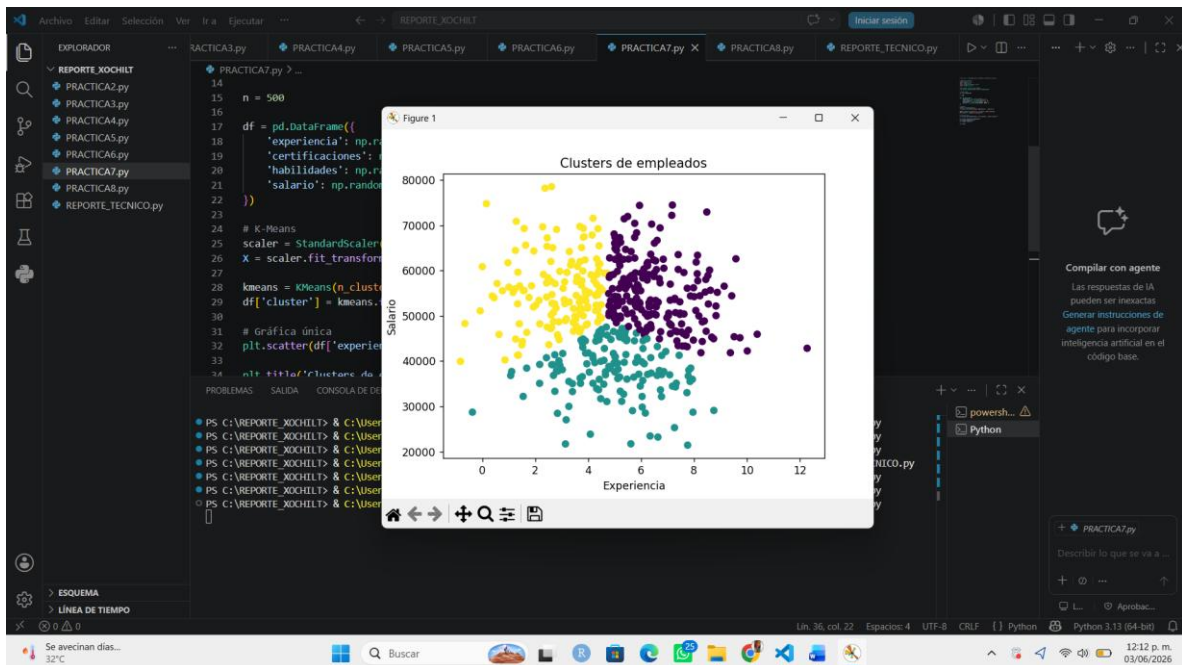
```
plt.title('Clusters de empleados')
```

```
plt.xlabel('Experiencia')
```

```
plt.ylabel('Salario')
```

```
plt.show()
```

Resultado:



Práctica 8: Clasificación de Intrusiones con KNN y Logística

Código:

```
# Práctica 8: Clasificación de Intrusiones con KNN y Logística
```

```
# Importar librerías
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.cluster import KMeans
```

```
# Crear datos
```

```
np.random.seed(444)
```

```
n = 400
```

```
df = pd.DataFrame({  
    'latencia': np.random.normal(20, 5, n),  
    'fallos': np.random.poisson(1, n),  
    'paquete': np.random.normal(500, 100, n)  
})
```

```
# K-Means
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(df)
```

```
kmeans = KMeans(n_clusters=2, n_init=10, random_state=42)
```

```
df['cluster'] = kmeans.fit_predict(X)
```

```
# Gráfica única
```

```
plt.scatter(df['latencia'], df['fallos'], c=df['cluster'])
```

```
plt.title('Clusters de red')
```

```
plt.xlabel('Latencia')
```

```
plt.ylabel('Fallos')
```

```
plt.show()
```

Resultado:

